Two Pointer Technique

Two pointers efficiently solve sorted array problems by using two indices that move toward/away from each other, eliminating $O(n^2)$ brute force.

Core Idea: Start with left=0, right=n-1. Compare values at pointers, then move the appropriate one based on comparison result.

Key Patterns:

- **Pair Sum: left++ if sum < target, right-- if sum > target (LC 167)**
- **Remove Duplicates: Slow pointer writes unique values, fast pointer scans (LC 26)**
- **Container Problems: Move shorter pointer (area limited by min height) (LC 11)**
- **Palindrome: Compare from both ends toward center (LC 125)**

Time: O(n), Space: O(1)

Binary Search (Variants)

Binary search halves search space each step: mid = left + (right-left)/2, then eliminate half based on nums[mid] ? target.

Standard: Find exact match (LC 704)

Variants:

- **Lower Bound: First position ≥ target (nums[mid] >= target ? right=mid : left=mid+1) (LC 35)**
- **Upper Bound: First position > target (nums[mid] > target ? right=mid : left=mid+1)**
- **First Occurrence: Modify condition to nums[mid] >= target (LC 34 left half)**
- **Last Occurrence: Modify to nums[mid] > target (LC 34 right half)**
- **Binary on Answer: Binary search optimal value (steps, time, etc.) instead of position**

Time: O(log n), Space: O(1)

Prefix Sum / Precomputation

Precompute cumulative sums to answer range queries in O(1) after O(n) preprocessing.

1D Formula: prefix[i] = prefix[i-1] + nums[i-1]
Range Sum: sum(l,r) = prefix[r+1] - prefix[l] (LC 303)

Key Applications:

- **Range sum queries (LC 303, 304 2D)**

- **Subarray sum = k using prefix[j] - prefix[i] = k + hashmap (LC 560)**
- **Product except self using left/right prefix products (LC 238)**

2D Formula: prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1] + matrix[i-1][j-1]

Time: O(n) preprocess + O(1) query, Space: O(n)

Basic Sorting (Conceptual Use)

Master when/why to sort rather than implementation details.

Key Insights:

- **Quick Sort: Good average O(n log n), O(n^2) worst, in-place**
- **Merge Sort: Guaranteed O(n log n), O(n) space, stable**
- **Dutch Flag: 3-way partition (LC 75: 0,1,2 colors)**
- **Sorting + Two Pointers: Sort first, then two pointers for triplets/pairs (LC 16)**

Pattern Recognition:

text

If problem asks "closest", "kth", "top", or has pairs/triplets → Sort first

Choose: Quickselect for kth largest (LC 215), Merge for external sort, Counting for small ranges.

Sliding Window (Fixed Size)

Maintain fixed-size window [i, i+k-1] that slides right, updating answer efficiently.

Core Steps:

1. **Compute initial window [0,k-1]**
2. **Slide: current = current - arr[i-k] + arr[i]**
3. **Track max/avg/count**

Variants:

text

Max Sum k: sum += arr[i] - arr[i-k]; max = Math.max(max,sum)

Max Element k: Deque maintains decreasing order (LC 239)

Distinct Count k: HashMap tracks frequency, size = distinct

Average k: Same as max sum / k (LC 643)

Time: O(n), Space: O(1) or O(k)

Pattern Summary Table

Topic	When to Use	Key Movement Rule	Example Problems
Two Pointers	Sorted arrays, pairs	Move based on sum comparison	167, 26, 11, 125
Binary Search	Sorted, find position/value	Eliminate half each step	704, 34, 35, 278
Prefix Sum	Range queries, subarray sums	prefix[r+1]-prefix[l]	303, 560, 238
Sorting	Kth, closest, triplets	Sort then pointers	75, 16, 215
Sliding Window	Fixed k subarrays	Slide: subtract left, add right	643, 239, Distinct

Pro Tip: Always ask "Is array sorted?" → Two pointers/binary. "Fixed window size k?" → Sliding window. "Multiple queries?" → Prefix sum.

Easy Level Solutions

Q1: Pair with Target Sum (Two Pointers)

Input: Sorted array, target → Output: "Pair Exists" or "Pair Not Exists"

java

```
import java.util.*;
public class Q1_PairSum {
    public static boolean pairSum(int[] arr, int target) {
        int left = 0, right = arr.length - 1;  // Two pointers at ends
        while (left < right) {
            int sum = arr[left] + arr[right];
```

```
        if (sum == target) return true;     // Found pair
        else if (sum < target) left++;      // Need larger sum
        else right--;                       // Need smaller sum
    }
    return false;
  }
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt(); int[] arr = new int[n];
    for(int i=0; i<n; i++) arr[i] = sc.nextInt();
    int target = sc.nextInt();
    System.out.println(pairSum(arr, target) ? "Pair Exists" : "Pair Not Exists");
  }
}
// Time: O(n), Space: O(1)
```

Sample Run: 1 2 4 7 11 15 15 → "Pair Exists" (11+4=15)

Q2: Maximum Sum Subarray of Size K (Sliding Window)

Input: Array, k → Output: Max sum

java

```
import java.util.*;
public class Q2_MaxSumK {
  public static int maxSumSubarray(int[] arr, int k) {
    int sum = 0;
    // Step 1: Initial window sum [0,k-1]
    for (int i = 0; i < k; i++) sum += arr[i];
    int maxSum = sum;

    // Step 2: Slide window
```

```
        for (int i = k; i < arr.length; i++) {
            sum += arr[i] - arr[i - k];     // Add new, remove old
            if (sum > maxSum) maxSum = sum;
        }
        return maxSum;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] arr = new int[n];
        for(int i=0; i<n; i++) arr[i] = sc.nextInt();
        int k = sc.nextInt();
        System.out.println(maxSumSubarray(arr, k));
    }
}
// Time: O(n), Space: O(1)
```

Sample Run: 2 1 5 1 3 2 7 4 3 → 12 (5+1+3=9? Wait, 1+3+2=6? Input 8 nums k=3: 5+1+3=9, but output 12? Check: 3+2+7=12 ✓)

Medium Level Solutions

Q3: First and Last Occurrence (Binary Search)

Input: Sorted array, key → Output: First & Last indices

java

```
import java.util.*;
public class Q3_FirstLast {
    public static int[] firstLast(int[] arr, int key) {
        int first = findFirst(arr, key);
        int last = findLast(arr, key);
        return new int[]{first, last};
    }
```

```
// Lower bound: first >= key
static int findFirst(int[] arr, int key) {
    int left = 0, right = arr.length;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] >= key) right = mid;   // Go left
        else left = mid + 1;
    }
    return (left < arr.length && arr[left] == key) ? left : -1;
}

// Upper bound - 1: last <= key
static int findLast(int[] arr, int key) {
    int left = 0, right = arr.length;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] > key) right = mid;    // Go left
        else left = mid + 1;
    }
    return (left - 1 >= 0 && arr[left - 1] == key) ? left - 1 : -1;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt(); int[] arr = new int[n];
    for(int i=0; i<n; i++) arr[i] = sc.nextInt();
    int key = sc.nextInt();
```

```
        int[] res = firstLast(arr, key);
        System.out.println("First = " + res[0] + " Last = " + res[1]);
    }
}
// Time: O(log n), Space: O(1)
```

Sample Run: 1 2 2 2 3 4 5 2 → "First = 1 Last = 3"

Q4: Prefix Sum – Range Query

Input: Array size n, then query l r → Output: Sum[l..r]

java

```
import java.util.*;
public class Q4_PrefixSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        // Build prefix sum
        int[] prefix = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            prefix[i] = prefix[i - 1] + arr[i - 1];
        }

        // Query (single for sample)
        int l = sc.nextInt(), r = sc.nextInt();
        System.out.println(prefix[r + 1] - prefix[l]);  // O(1) query
    }
}
```

// Time: O(n) build + O(1) query

Sample Run: 1 3 5 7 9 query 1 3 → 15 (3+5+7)

Q5: Remove Duplicates from Sorted Array

Input: Sorted array → Output: New length & first 'length' elements

java

```
import java.util.*;
public class Q5_RemoveDuplicates {
    public static int removeDuplicates(int[] arr) {
        if (arr.length == 0) return 0;
        int write = 0;  // Position to write next unique
        for (int read = 1; read < arr.length; read++) {
            if (arr[read] != arr[write]) {
                arr[++write] = arr[read];  // Copy unique
            }
        }
        return write + 1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] arr = new int[n];
        for(int i=0; i<n; i++) arr[i] = sc.nextInt();
        int len = removeDuplicates(arr);
        System.out.print("Length = " + len);
        System.out.print(" Array = ");
        for (int i = 0; i < len; i++) System.out.print(arr[i] + " ");
    }
}
```

// Time: O(n), Space: O(1) in-place

Sample Run: 1 1 2 2 3 4 4 → "Length = 4 Array = 1 2 3 4 "

Hard Level Solution

Q6: Longest Subarray with Sum ≤ K (Sliding Window + Prefix)

Note: Uses variable sliding window (two pointers) for max length where sum ≤ k.

java

```
import java.util.*;
public class Q6_LongestSubarray {
    public static int longestSubarray(int[] arr, int k) {
        int left = 0, sum = 0, maxLen = 0;

        for (int right = 0; right < arr.length; right++) {
            sum += arr[right];  // Expand window

            // Shrink from left until sum <= k
            while (sum > k && left <= right) {
                sum -= arr[left];
                left++;
            }

            // Update max length
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); int[] arr = new int[n];
        for(int i=0; i<n; i++) arr[i] = sc.nextInt();
```

```
        int k = sc.nextInt();
        System.out.println(longestSubarray(arr, k));
    }
}
// Time: O(n), Space: O(1)
```

Sample Run: 4 2 1 7 8 1 2 8 8 → 3 (e.g., =7≤8)

1-5: Two Pointer Technique

1. LC 167: Two Sum II - Input Array Is Sorted [Easy]

java

```
import java.util.*;

public class LC167 {
    public static int[] twoSum(int[] numbers, int target) {
        int left = 0, right = numbers.length - 1;
        while (left < right) {
            int sum = numbers[left] + numbers[right];
            if (sum == target) return new int[]{left+1, right+1};
            else if (sum < target) left++;
            else right--;
        }
        return new int[]{};
    }
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in); int n = sc.nextInt(); int[] nums = new int[n];
        for(int i=0;i<n;i++) nums[i]=sc.nextInt(); int target=sc.nextInt();
        System.out.println(Arrays.toString(twoSum(nums,target)));
    }
}
// O(n) time, O(1) space [web:11]
```

2. LC 26: Remove Duplicates from Sorted Array [Easy]

java

```
import java.util.*;

public class LC26 {
    public static int removeDuplicates(int[] nums) {
        if(nums.length==0) return 0;
        int i=0;
        for(int j=1;j<nums.length;j++){
            if(nums[j]!=nums[i]) nums[++i]=nums[j];
        }
        return i+1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); int n = sc.nextInt(); int[] nums = new int[n];
        for(int i=0;i<n;i++) nums[i]=sc.nextInt();
        int len = removeDuplicates(nums);
        System.out.println(len + " " + Arrays.toString(Arrays.copyOf(nums,len)));
    }
}
// O(n) time, O(1) space [web:12]
```

3. LC 11: Container With Most Water [Medium]

java

```java
public class LC11 {
    public static int maxArea(int[] height) {
        int left=0, right=height.length-1, max=0;
        while(left<right){
            int area = Math.min(height[left],height[right])*(right-left);
            max = Math.max(max,area);
            if(height[left]<height[right]) left++;
            else right--;
        }
        return max;
    }
    // Add main similar to above
}
// O(n) time, O(1) space [web:13]
```

4. LC 125: Valid Palindrome [Easy]

java

```java
public class LC125 {
    public static boolean isPalindrome(String s) {
        s = s.toLowerCase().replaceAll("[^a-z0-9]","");
        int left=0, right=s.length()-1;
        while(left<right){
            if(s.charAt(left++) != s.charAt(right--)) return false;
        }
        return true;
    }
}
// O(n) time, O(1) space [web:19]
```

5. LC 283: Move Zeroes [Easy]

java

```
public class LC283 {
    public static void moveZeroes(int[] nums) {
        int i=0;
        for(int j=0;j<nums.length;j++){
            if(nums[j]!=0) nums[i++]=nums[j];
        }
        while(i<nums.length) nums[i++]=0;
    }
}
// O(n) time, O(1) space
```

6-10: Binary Search Variants

6. LC 704: Binary Search [Easy]

java

```
public class LC704 {
    public static int search(int[] nums, int target) {
        int left=0, right=nums.length-1;
        while(left<=right){
            int mid = left + (right-left)/2;
            if(nums[mid]==target) return mid;
            else if(nums[mid]<target) left=mid+1;
            else right=mid-1;
        }
        return -1;
    }
}
// O(log n) time, O(1) space [web:15]
```

7. LC 34: Find First and Last Position [Medium]

java

```java
public class LC34 {
    public static int[] searchRange(int[] nums, int target) {
        int first = firstPos(nums,target), last = lastPos(nums,target);
        return first==-1 ? new int[]{-1,-1} : new int[]{first,last};
    }
    static int firstPos(int[] nums,int target){
        int l=0,r=nums.length;
        while(l<r){
            int m=l+(r-l)/2;
            if(nums[m]>=target) r=m;
            else l=m+1;
        }
        return l<nums.length && nums[l]==target ? l : -1;
    }
    static int lastPos(int[] nums,int target){
        int l=0,r=nums.length;
        while(l<r){
            int m=l+(r-l)/2;
            if(nums[m]>target) r=m;
            else l=m+1;
        }
        return l-1>=0 && nums[l-1]==target ? l-1 : -1;
    }
}
// O(log n) time, O(1) space [web:14]
```

8. LC 35: Search Insert Position [Easy]

java

```
public class LC35 {
    public static int searchInsert(int[] nums, int target) {
        int l=0,r=nums.length;
        while(l<r){
            int m=l+(r-l)/2;
            if(nums[m]<target) l=m+1;
            else r=m;
        }
        return l;
    }
}
// O(log n) time, O(1) space
```

9. LC 278: First Bad Version [Easy]

java

```
/* The isBadVersion API is defined in the parent class Better. */
public class LC278 extends VersionControl {
    public int firstBadVersion(int n) {
        int l=1,r=n;
        while(l<r){
            int m=l+(r-l)/2;
            if(isBadVersion(m)) r=m;
            else l=m+1;
        }
        return l;
    }
}
// O(log n) time
```

10. LC 1620: Coordinate With Maximum Freedom Score (Binary Search on Answer) [Medium]

java

```
public class LC1620 {
   public static int maxScore(int[][] points, int k) {
      // Simplified: binary search optimal steps; actual impl uses prefix dist
      int l=0, r=1000000; // Binary on score
      while(l<r){
         int m=(l+r+1)/2;
         if(canAchieve(points,k,m)) l=m;
         else r=m-1;
      }
      return l;
   }
   // canAchieve logic omitted for brevity; checks if score m reachable
}
// O(n log MAX) [web:6]
```

11-15: Prefix Sum / Precomputation

11. LC 303: Range Sum Query Immutable [Easy]

java

```
class NumArray {
   int[] prefix;
   public NumArray(int[] nums) {
      prefix = new int[nums.length+1];
      for(int i=1;i<prefix.length;i++) prefix[i]=prefix[i-1]+nums[i-1];
   }
   public int sumRange(int left, int right) {
      return prefix[right+1]-prefix[left];
```

```
    }
}
// O(n) build, O(1) query [web:7]
```

12. LC 560: Subarray Sum Equals K [Medium]

java

```
import java.util.*;
public class LC560 {
    public static int subarraySum(int[] nums, int k) {
        Map<Integer,Integer> map = new HashMap<>(); map.put(0,1);
        int prefix=0, count=0;
        for(int num:nums){
            prefix+=num;
            count+=map.getOrDefault(prefix-k,0);
            map.put(prefix,map.getOrDefault(prefix,0)+1);
        }
        return count;
    }
}
// O(n) time, O(n) space [web:16]
```

13. LC 304: Range Sum Query 2D [Medium]

java

```
class NumMatrix {
    int[][] prefix;
    public NumMatrix(int[][] matrix) {
        int m=matrix.length, n=matrix[0].length;
        prefix = new int[m+1][n+1];
        for(int i=1;i<=m;i++)
            for(int j=1;j<=n;j++)
```

```
            prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1] + matrix[i-1][j-1];
    }
    public int sumRegion(int row1, int col1, int row2, int col2) {
        return prefix[row2+1][col2+1] - prefix[row1][col2+1] - prefix[row2+1][col1] +
prefix[row1][col1];
    }
}
// O(mn) build, O(1) query
```

14. LC 437: Path Sum III [Medium]

java

```
// Tree prefix sum variant; similar hashmap logic as 560
// O(n) time with recursion + map
```

15. LC 238: Product of Array Except Self [Medium]

java

```
public class LC238 {
    public static int[] productExceptSelf(int[] nums) {
        int n=nums.length, suffix=1, prefix=1;
        int[] res = new int[n];
        res[n-1]=1;
        for(int i=n-2;i>=0;i--) res[i]=suffix*=nums[i+1]; // Right prefix
        for(int i=1;i<n;i++){ prefix*=nums[i-1]; res[i]*=prefix; }
        return res;
    }
}
// O(n) time, O(1) space
```

16-20: Basic Sorting (with Pointers/Special)

16. LC 75: Sort Colors [Medium]

java

```
public class LC75 {
    public static void sortColors(int[] nums) {
        int low=0, mid=0, high=nums.length-1;
        while(mid<=high){
            if(nums[mid]==0) swap(nums,low++,mid++);
            else if(nums[mid]==1) mid++;
            else swap(nums,mid,high--);
        }
    }
    static void swap(int[] a,int i,int j){ int t=a[i];a[i]=a[j];a[j]=t; }
}
// O(n) time, O(1) space
```

17. LC 88: Merge Sorted Array [Easy]

java

```
public class LC88 {
    public static void merge(int[] nums1, int m, int[] nums2, int n) {
        int i=m-1,j=n-1,k=m+n-1;
        while(i>=0 && j>=0){
            nums1[k--] = nums1[i]>nums2[j] ? nums1[i--] : nums2[j--];
        }
        while(j>=0) nums1[k--]=nums2[j--];
    }
}
// O(m+n) time, O(1) space
```

18. LC 16: 3Sum Closest (Sort + Two Pointers) [Medium]

java

```
import java.util.*;
public class LC16 {
```

```
    public static int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int closest = nums[0]+nums[1]+nums[2];
        for(int i=0;i<nums.length-2;i++){
            int l=i+1,r=nums.length-1;
            while(l<r){
                int sum=nums[i]+nums[l]+nums[r];
                if(Math.abs(sum-target)<Math.abs(closest-target)) closest=sum;
                if(sum<target) l++;
                else r--;
            }
        }
        return closest;
    }
}
// O(n^2) time, O(1) space
```

19. LC 977: Squares of a Sorted Array [Easy]

java

```
public class LC977 {
    public static int[] sortedSquares(int[] nums) {
        int[] res = new int[nums.length], l=0, r=nums.length-1, p=r;
        while(l<=r){
            int ls=nums[l]*nums[l], rs=nums[r]*nums[r];
            if(ls>rs) res[p--]=ls++;
            else res[p--]=rs++;
        }
        return res;
    }
```

```
}
// O(n) time, O(n) space
```

20. LC 215: Kth Largest Element (Quickselect Sort Variant) [Medium]

java

```
// Quickselect implementation for conceptual clarity
public class LC215 {
    public static int findKthLargest(int[] nums, int k) {
        return quickselect(nums,0,nums.length-1,k-1);
    }
    static int quickselect(int[] a,int l,int r,int k){
        if(l==r) return a[l];
        int p=partition(a,l,r), rank=p-l;
        if(k==rank) return a[p];
        return k<rank ? quickselect(a,l,p-1,k) : quickselect(a,p+1,r,k-rank-1);
    }
    static int partition(int[] a,int l,int r){
        int pivot=a[r], i=l-1;
        for(int j=l;j<r;j++) if(a[j]<=pivot) swap(a,++i,j);
        swap(a,++i,r); return i;
    }
    static void swap(int[] a,int i,int j){int t=a[i];a[i]=a[j];a[j]=t;}
}
// Avg O(n), worst O(n^2)
```

21-25: Sliding Window Fixed Size

21. LC 643: Maximum Average Subarray I [Easy]

java

```
public class LC643 {
    public static double findMaxAverage(int[] nums, int k) {
```

```
        double sum=0;
        for(int i=0;i<k;i++) sum+=nums[i];
        double max=sum;
        for(int i=k;i<nums.length;i++){
            sum+=nums[i]-nums[i-k];
            max=Math.max(max,sum);
        }
        return max/k;
    }
}
// O(n) time, O(1) space
```

22. Max Sum Subarray of Size K (GfG/Adapt LC)

java

```
public class MaxSumK {
    public static int maxSum(int[] arr, int k) {
        int sum=0;
        for(int i=0;i<k;i++) sum+=arr[i];
        int max=sum;
        for(int i=k;i<arr.length;i++){
            sum+=arr[i]-arr[i-k];
            if(sum>max) max=sum;
        }
        return max;
    }
}
// O(n) [web:9]
```

23. LC 239: Sliding Window Maximum [Hard]

java

```java
import java.util.*;
public class LC239 {
    public static int[] maxSlidingWindow(int[] nums, int k) {
        Deque<Integer> dq = new ArrayDeque<>();
        int[] res = new int[nums.length-k+1];
        for(int i=0;i<nums.length;i++){
            while(!dq.isEmpty() && nums[dq.peekLast()]<=nums[i]) dq.pollLast();
            dq.offerLast(i);
            if(dq.peekFirst()==i-k) dq.pollFirst();
            if(i>=k-1) res[i-k+1]=nums[dq.peekFirst()];
        }
        return res;
    }
}
// O(n) time, O(k) space [web:17]
```

24. Count Distinct in Window Size K (GfG Adapt)

java

```java
import java.util.*;
public class DistinctWindow {
    public static int[] countDistinct(int[] arr, int k) {
        int[] res = new int[arr.length-k+1];
        Map<Integer,Integer> map = new HashMap<>();
        for(int i=0;i<k;i++) map.put(arr[i],map.getOrDefault(arr[i],0)+1);
        res[0]=map.size();
        for(int i=k;i<arr.length;i++){
            map.put(arr[i-k],map.get(arr[i-k])-1);
            if(map.get(arr[i-k])==0) map.remove(arr[i-k]);
            map.put(arr[i],map.getOrDefault(arr[i],0)+1);
```

```
            res[i-k+1]=map.size();
        }
        return res;
    }
}
// O(n) time, O(k) space [web:10]
```

25. LC 219: Contains Duplicate II (Fixed Window Variant) [Easy]

java

```
import java.util.*;
public class LC219 {
    public static boolean containsNearbyDuplicate(int[] nums, int k) {
        Set<Integer> set = new HashSet<>();
        for(int i=0;i<nums.length;i++){
            if(set.contains(nums[i])) return true;
            set.add(nums[i]);
            if(set.size()>k) set.remove(nums[i-k]);
        }
        return false;
    }
}
// O(n) time, O(k) space
```